

Motorcycle Fleet Garage

Workshop software for a garage that maintains delivery and courier motorcycle fleets: technician phone app, manager dashboard and client portal

IN ONE SENTENCE

Keeps courier motorcycle fleets on the road: technicians work from their phones (even offline), managers run jobs, parts and invoices, and each fleet client sees its own bikes.

BUILT WITH

Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	7
5. Build it from scratch, step by step	9
6. Testing, deployment and known limits	13

1. What this project is

Delivery companies (food delivery, couriers, parcel firms) run fleets of motorcycles that ride all day. Every hour a bike is off the road costs them money, so they contract a garage to service and repair the fleet quickly and to report on it.

This project is that garage's system, in three parts: a **technician phone app** (/tech) that works without mobile signal, a **manager dashboard** for jobs, bikes, parts, invoicing and reports, and a **client portal** where each fleet sees only its own bikes, reports problems and reads its invoices.

At a glance	
Type	Web app with an offline-first phone view for technicians
Users	Technicians, workshop manager, admin, fleet clients
Stack	Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker
Tests	12 shared rule tests, 22 API tests on a real database, 8 web tests, browser smoke test including losing signal mid-job

Who uses it

User	What they do with it
Technician	Picks up jobs on the phone, runs the job clock, fills the checklist, adds parts, notes and photos, signs off with the odometer.
Manager	Books jobs, assigns technicians, corrects time, manages bikes, parts and stock, runs invoicing, reads reports.
Admin	Everything above plus clients, rates, settings, users and the activity log.
Fleet client	Sees its own bikes and their service status, reports problems ("unsafe to ride"), follows jobs and reads invoices.

2. The business problem

A garage servicing several courier fleets juggles hundreds of bikes, each with its own service interval, and clients who want every bike back on the road today. Technicians write times and parts on paper, so labour is under-billed and stock disappears. The workshop floor often has poor mobile signal. Clients phone in problems and ask "which of my bikes are off the road?" all day.

The client wants a system where **time and parts are captured on the job, on the phone, even offline**, bikes that fail safety checks can't slip back onto the road, invoicing per client is accurate and fast, and fleet clients can serve themselves.

Pain point today	What it costs the business	How this project solves it
Time and parts written on paper	Labour under-billed, stock losses	Technicians clock time and add parts on their phone; stock moves are recorded with who and which job
No signal in the workshop	Apps fail, technicians give up	Offline-first: actions are saved on the phone and sent in order when signal returns, never applied twice
Services missed	Breakdowns, unhappy clients	Service due by distance or time, whichever is first; a due list on the dashboard
Unsafe bikes go back out	Accidents, liability	A failed safety check keeps the bike off the road with an automatic urgent follow-up repair
Invoicing per client takes days	Late cash, billing errors	Invoice run per client and period with preview; gapless numbers; a job can't be billed twice
Clients phone for status	Staff time lost	Client portal with their bikes, jobs, off-road status and invoices; problem reporting

Value to the business

- More billable hours captured, less stock shrinkage, faster invoicing.
- Fleet availability reports prove the garage's service level to its clients.
- Safety rules enforced by the system, with a full audit trail.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Technician app	See my jobs and unassigned jobs; starting the clock on an unassigned job takes it.
FR-02	Technician app	One running clock per technician: starting a job stops the previous one.
FR-03	Technician app	Pass / Fail / N/A checklist; a failure needs a note.
FR-04	Technician app	Add parts from stock, write notes, take photos, mark "waiting for parts", sign off with the odometer reading.
FR-05	Technician app	Work offline: every action is saved on the phone and sent in order with its own id when signal returns; refused actions show under "Not saved" with the reason.
FR-06	Jobs	Jobs of type service, inspection, repair or defect, with priority, booking date and technician.
FR-07	Jobs	Time entries can be corrected until the job is invoiced; every correction is logged.
FR-08	Bikes	Bikes per client with registration, model and odometer; odometer readings can't go backwards.
FR-09	Bikes	Service due by distance or time, whichever comes first.
FR-10	Bikes	A failed safety check keeps the bike off the road under an automatic urgent follow-up repair.
FR-11	Parts	Deliveries, returns and stock-takes; stock can't go negative; move history shows who used what on which job.
FR-12	Invoicing	Invoice per client and period with a preview: labour at the client's agreed rate (or default) in configurable time blocks, plus parts and tax.
FR-13	Invoicing	Gapless numbering; a job can't be billed twice; voiding keeps the number and frees the jobs.
FR-14	Reports	Fleet availability per client (off-road hours merged per bike), technician hours, parts used, turnaround; each exports to CSV.
FR-15	Client portal	Clients see only their bikes, service status and what is off the road.
FR-16	Client portal	Report a problem, flagging "unsafe to ride" to make it urgent and off the road.
FR-17	Client portal	Follow jobs, read the work done and see invoices; never see part costs, workshop notes or other clients.

Non-functional requirements

ID	Quality	Requirement
NFR-01	Security	Passwords hashed with bcrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.

ID	Quality	Requirement
NFR-02	Security	Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https).
NFR-03	Security	Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.
NFR-04	Security	An activity (audit) log of logins and every business action, viewable by admins.
NFR-05	Security	Client scoping is enforced in every query; uploads are checked by their real bytes; CSV exports are protected against formula injection.
NFR-06	Reliability offline	Replaying the same action twice has no effect (processed operation ids are stored); actions are applied in order; conflicts are reported, not lost.
NFR-07	Integrity	Database rules: one running clock per technician, registrations unique among bikes in service, stock never below zero, a client login belongs to exactly one client, invoice totals must add up.
NFR-08	Integrity	Gapless job and invoice numbers; optimistic locking stops two people overwriting each other's edits.
NFR-09	Usability	The technician view is built for a phone in one hand: large buttons, few fields, works in dark mode.
NFR-10	Operations	Runs as one Docker image next to PostgreSQL; health checks /healthz and /readyz ; database migrations run on start-up safely with several instances.
NFR-11	Operations	Structured JSON logs with a request id (returned in the x-request-id header); cookies and tokens are redacted.

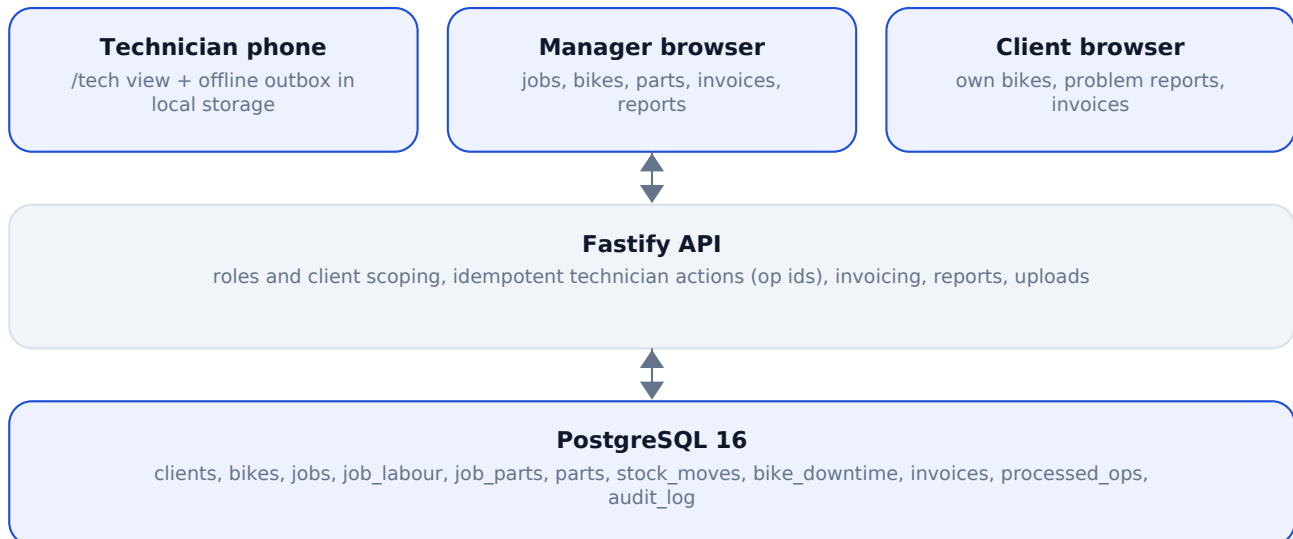
Out of scope for this release

- Online card payments for invoices.
- Native mobile apps (the web app is used on phones instead).
- Push notifications to clients.

4. How the solution is designed

One server and database serve three front-ends in the same React app: manager pages, the technician phone view with its offline outbox, and the client portal. The server decides what each role may see; the client portal simply gets less data.

Architecture



Technology choices

Part	Choice	Why
Server	Node 22 + Fastify 5	Fast, typed; easy to add idempotency and role checks per route.
Database	PostgreSQL 16	Partial unique indexes enforce one running clock and one open downtime per job.
Offline	Outbox in the browser (web/src/lib/outbox.ts)	Queued operations with ids, sent in order, retried; no extra framework needed.
Shared rules	shared/fleet.ts	Service due, checklist problems, billable minutes, invoice lines and totals, downtime hours as pure functions.
Web app	React 19, TanStack Query	One app, three experiences; phone-first technician view.
Tests	Vitest, Playwright	A browser test goes offline mid-job and checks the sync afterwards.

Data model

Table / file	What it holds
clients / users	Fleet clients with agreed labour rate; users with role and, for client logins, their client.
bikes / bike_downtime	Bikes with odometer and service data; off-road periods per job (one open per job).
jobs / job_labour / job_parts	Jobs with type, priority, status, checklist; time entries (one running per technician); parts used.
parts / stock_moves	Parts with stock level (never negative); every movement with reason, user and job.

Table / file	What it holds
job_notes / job_photos	Technician notes and photos (bytes checked by content).
invoices / invoice_lines / counters	Invoices per client and period, their lines, gapless numbers.
processed_ops	Ids of technician actions already applied, so a replay does nothing.
settings / sessions / audit_log	Rates, blocks, checklist template; sessions; activity log.

Key business rules

- **Service due** when either the distance since the last service or the time since it passes the interval, whichever comes first.
- **Billable labour:** minutes rounded up to the configured block (e.g. 15 min), times the client's hourly rate.
- **Failed safety check:** sign-off creates an urgent follow-up repair job and keeps the bike's downtime open until it passes.
- **Fleet availability:** off-road intervals per bike are merged before summing, so overlapping jobs are not double-counted.

Folder layout

```

shared/  fleet.ts (rules) · schemas.ts · types.ts · csv.ts
server/  src/routes (tech, jobs, fleet, billing, reports, admin, auth) · migrations/001_init.
  sql
web/     src/pages (Dashboard, Jobs, JobPage, Bikes, Parts, Invoices, Reports, Clients, Tech...)
        src/lib/outbox.ts (offline queue) · scripts/smoke.mjs

```

5. Build it from scratch, step by step

Start with the data and the manager screens, then add the technician phone view, then make it offline-first, then invoicing and the client portal.

Before you start

- Node.js 22+, PostgreSQL 16, Git.
- A phone (or Chrome's device mode) to try the technician view.

Step 1. Set up the workspace

One repository with a server, a web app and shared code used by both.

- 1.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir motorcycle-fleet-garage && cd motorcycle-fleet-garage && git init
npm init -y # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg
zod @fastify/multipart
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

- 1.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

Done when: `npm run typecheck` passes on the empty project.

Step 2. Configuration, database connection and migrations

- 2.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`) and refuse to start with a clear message when one is wrong.
- 2.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.
- 2.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

Done when: Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

Step 3. Design the database schema

- 3.1 Create the tables in the data model with foreign keys and CHECK constraints (stock $\geq$ 0, quantity > 0, invoice totals add up).

```
CREATE UNIQUE INDEX job_labour_one_running ON job_labour (technician_id) WHERE ended_at IS NULL;  
CREATE UNIQUE INDEX bikes_plate_live_key ON bikes (plate) WHERE NOT retired;  
CREATE UNIQUE INDEX bike_downtime_one_open ON bike_downtime (job_id) WHERE ended_at IS NULL;
```

Done when: The schema migrates and the constraints reject bad rows in a quick manual test.

Step 4. Logins, roles and request security

The same proven foundation every production app here uses.

- 4.1 **Passwords:** hash with Node's built-in `scrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 4.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (Secure in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 4.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 4.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 4.5 **Roles:** technician (own and unassigned jobs), manager (workshop), admin (settings, users) and client (own bikes only, scoped in every query). A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 4.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 4.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

Done when: API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

Step 5. Shared fleet rules

- 5.1 Write `serviceDue()`, `checklistProblems()`, `billableMinutes()`, `labourCents()`, `invoiceLines()`, `invoiceTotals()` and `coveredHours()` (merged downtime) in `shared/fleet.ts`.
- 5.2 Unit-test the boundaries: exactly at the interval, block rounding, overlapping downtime.

Done when: Shared tests pass.

Step 6. Bikes, clients and parts

- 6.1 API and pages for clients, bikes (odometer never goes backwards) and parts with deliveries, returns and stock-takes, each writing a `stock_moves` row inside the same transaction as the stock change.

Done when: Stock can't go negative; every change has a move record.

Step 7. Jobs and the job clock

- 7.1 Jobs with type, priority, booking date, technician and checklist; manager pages to list, filter and edit.
- 7.2 Clock start/stop: starting a job stops the technician's running entry first (the unique index is the safety net).
- 7.3 Sign-off: checklist complete, failures noted, odometer recorded; a failed safety item creates an urgent follow-up job and keeps the bike off the road.

Done when: Tests: one clock per technician; sign-off rules; follow-up job created.

Step 8. Technician phone view

- 8.1 `/tech`: my jobs and unassigned jobs, big buttons for clock, checklist, parts, notes, photos, waiting-for-parts and sign-off.
- 8.2 Every action gets a client-generated id; the server stores it in `processed_ops` inside the same transaction, so a repeat returns success without doing anything.

Done when: Replaying the same request twice changes nothing the second time (API test).

Step 9. Make it work offline

- 9.1 `outbox.ts`: store pending actions in local storage per user; send them one at a time in order when online; keep failed ones with the server's reason under "Not saved".
- 9.2 Show local changes immediately; refresh from the server after each sync.

Done when: In the browser test, the phone goes offline, finishes a job, comes back online and the job is signed off on the server.

Step 10. Invoicing

- 10.1 Preview an invoice for a client and period from finished, unbilled jobs (labour blocks at the client's rate, parts, tax).
- 10.2 Issue in one transaction: take the next number, write lines and totals, mark jobs billed. Void keeps the number and frees the jobs.
- 10.3 Printable invoice page.

Done when: Tests: a job can't be billed twice; void frees jobs; numbers are gapless.

Step 11. Client portal and reports

- 11.1 Client pages: their bikes with service status and off-road flags, problem reporting ("unsafe to ride" makes it urgent and off the road), jobs and invoices; costs and workshop notes removed from responses.
- 11.2 Reports: availability per client, technician hours, parts used, turnaround; CSV exports.

Done when: Tests: a client can't read another client's bike, job or invoice.

Step 12. Web app shell

- 12.1 `api/client.ts`: a small `fetch` wrapper that sends the CSRF token on writes, turns error responses into readable messages (with per-field details for forms), and sends the user to sign-in on 401.
- 12.2 `api/auth.ts`: TanStack Query hooks for the current user, sign-in and sign-out; a layout with navigation that shows only what the role may use.
- 12.3 Styles with light and dark mode and a phone layout (tables scroll inside their card, forms stack to one column).

Done when: You can sign in, see the navigation for your role and sign out.

Step 13. Demo data

- 13.1 `seed.ts`: clients, bikes, technicians, parts, jobs in every state, deliveries booked so stock never runs out, invoices; guarded by `ALLOW_DEMO_SEED`.

Done when: Every demo login shows meaningful data.

Step 14. Test it end to end

- 14.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 14.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 14.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then run a manager's day, a technician on a phone losing signal mid-job and syncing afterwards, a client reporting a problem, an invoice run and phone layouts. Fail on any script error, and save screenshots.

```
npm run typecheck && npm test
npm run build && npm run smoke
```

Done when: All three suites pass locally.

Step 15. Package, automate and document

- 15.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 15.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 15.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.
- 15.4 `README.md` for users and `docs/OPERATIONS.md` for whoever runs it (https, backups, monitoring, upgrades).

Done when: CI is green on GitHub and `docker compose up -d --build` gives a working app.

6. Testing, deployment and known limits

How it is tested

- **12 shared tests:** service due, checklist, labour billing, downtime merging.
- **22 API tests on PostgreSQL:** access control and client scoping, idempotent replay, one clock per technician, sign-off rules and follow-ups, stock, invoicing and voiding, uploads, per-session rate limits.
- **Browser smoke test:** manager's day, offline technician, client report, invoice run, phone layouts.
- Bugs found and fixed during the build included labour minutes differing between the job page and the invoice, and moving a bike to another client while it had open jobs.

```
npm run typecheck
npm test
npm run build && npm run smoke
```

Deploying and running it

```
cp .env.example .env # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email you@garage.com --name "Your Name"
# demo: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js
# logins admin@, manager@, tech@, client@demo.local • password demo-password-1
```

- Use https in production (phones on public networks), and back up the database including photos.

Known limits

- No online payments.
- Photos are stored in the database (move to object storage at scale).
- Rate limits are counted per server instance.

Sensible next steps

- Push notifications or WhatsApp messages to clients when a bike is ready.
- Parts ordering from suppliers when stock is low.
- A native app wrapper if technicians need camera or barcode features beyond the browser.